はい、承知いたしました。これまでの「物理基礎」「プーリング」「探索（BFS）」「状態管理（FSM）」に加え、コーディングテストで**「アルゴリズムの思考力」や「データ構造の応用」**が問われる可能性のある分野を、さらに2点追加で提示します。

---

5. ? 貪欲法（Greedy Algorithm）の基礎

ゲームロジックの中には、動的計画法（DP）ほど複雑ではないが、「その場での最適な選択」を繰り返すことで全体最適を目指す貪欲法が適用できる問題が出やすいです。

予想される問題

- リソース配分/スケジューリング:
  - 限られたリソース（例：資金、時間、行動回数）の中で、最も効率の良いアイテムや行動を優先的に選択し、最大のリターン（スコア、ダメージ）を得るロジックの実装。
  - 例：「ユニットを配置できるコストがCの時、配置コスト効率（ダメージ/コスト）が最大のユニットから順に配置せよ。」
- 区間スケジューリング:
  - 複数のイベント（スキル使用、ユニットの生産時間など）が与えられたとき、時間が重複しないように最大数のイベントを選択する問題。

対策の焦点

- ソートの基準: 貪欲法はソートが鍵となるため、問題の要求（例：「コスト効率」、「終了時間の早さ」）に基づいて、最適なソート基準を即座に定義できるか。
- C++での実装: `std::sort` とカスタムコンパレータ（ラムダ式など）を使って、配列やベクターを特定の基準でソートする練習。

C++ 実装例（コスト効率順の配置）

C++

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Unit {
  std::string name;
  int damage;
  int cost;
  float efficiency; // ダメージ / コスト

  void calculateEfficiency() {
    efficiency = (float)damage / cost;
  }
};

/**
```

```
 * 貪欲法:コスト効率が高い順にユニットを配置し、最大ダメージを計算する
 * @param availableUnits 配置可能な全ユニット
 * @param maxCost 利用可能な最大コスト
 * @return 敵に与えられる最大ダメージ
 */
int greedyPlacement(std::vector<Unit>& availableUnits, int maxCost) {
  // 1. 貪欲法の鍵: 効率(efficiency)が高い順にソートする
  std::sort(availableUnits.begin(), availableUnits.end(),
    [](const Unit& a, const Unit& b) {
      return a.efficiency > b.efficiency; // 降順
    });

  int currentCost = 0;
  int totalDamage = 0;

  std::cout << "--- 貪欲法によるユニット配置 ---" << std::endl;
  for (const auto& unit : availableUnits) {
    if (currentCost + unit.cost <= maxCost) {
      // 配置し、リソースを消費する
      currentCost += unit.cost;
      totalDamage += unit.damage;
      std::cout << unit.name << "を配置 (コスト: " << unit.cost
           << ", 効率: " << unit.efficiency << ")" << std::endl;
    } else {
      // コスト超過のため配置しない
      std::cout << unit.name << "はコスト超過のためスキップ" << std::endl;
    }
  }
  std::cout << "総コスト: " << currentCost << ", 総ダメージ: " << totalDamage <<
std::endl;
  return totalDamage;
}

// int main() {
//     std::vector<Unit> units = {
//         {"Archer", 50, 20}, // 効率 2.5
//         {"Knight", 120, 50}, // 効率 2.4
//         {"Wizard", 200, 80} // 効率 2.5
//     };
//     for (auto& unit : units) {
//         unit.calculateEfficiency();
//     }
//     greedyPlacement(units, 100);
//     return 0;
// }
```

6.? データ構造の応用:連結リストとポインタ操作

Unreal Engine の C++コード（特に TArray や TMap）では隠蔽されていますが、C++の基礎としてポインタとメモリ管理の理解は重要です。特に連結リストの操作は、ポインタの

扱いの正確性を測る良い指標となります。

予想される問題

- 単方向連結リストの操作:
  - リストの先頭、中間、末尾への要素の挿入、またはリストからの要素の削除を正確に行えるか。
  - 例：特定の条件を満たす敵ユニットをリストから削除するロジック。
- ポインタのエラー処理:
  - NULL ポインタチェック（`if (ptr != nullptr)`または`if (ptr)`）を確実に実行できるか。

対策の焦点

- 挿入/削除のロジック: リストの要素を操作する際、前後のポインタの接続順序を間違えないこと（特に削除時はメモリリークの回避）。

C++ 実装例（単方向連結リストからのノード削除）

C++

```
#include <iostream>

struct Node {
  int data;
  Node* next;
  Node(int d) : data(d), next(nullptr) {}
};

/**
 * リストから特定の値を持つ最初のノードを削除する
 * @param head リストの先頭ポインタへの参照
 * @param key 削除対象の値
 */
void deleteNode(Node*& head, int key) {
  Node* temp = head;
  Node* prev = nullptr;

  // 1. 先頭ノードが削除対象の場合
  if (temp != nullptr && temp->data == key) {
    head = temp->next; // headを次のノードへ移動
    delete temp;       // 元の先頭ノードを解放
    return;
  }

  // 2. 先頭ノード以外から削除対象を探す
  while (temp != nullptr && temp->data != key) {
    prev = temp;
    temp = temp->next;
  }
```

```
  // 3. 値が見つからなかった場合
  if (temp == nullptr) return;

  // 4. ノードをリストから切り離す (prevとtemp->nextを繋ぐ)
  prev->next = temp->next;

  // 5. ノードを解放する (メモリリーク防止)
  delete temp;
}

void printList(Node* node) {
  while (node != nullptr) {
    std::cout << node->data << " -> ";
    node = node->next;
  }
  std::cout << "NULL" << std::endl;
}

void testLinkedList() {
  Node* head = new Node(10);
  head->next = new Node(20);
  head->next->next = new Node(30);

  std::cout << "オリジナルリスト: ";
  printList(head); // 10 -> 20 -> 30 -> NULL

  // 20を削除
  deleteNode(head, 20);
  std::cout << "20を削除後:     ";
  printList(head); // 10 -> 30 -> NULL

  // 10(先頭)を削除
  deleteNode(head, 10);
  std::cout << "10を削除後:     ";
  printList(head); // 30 -> NULL

  // 存在しない値(50)を削除(処理なし)
  deleteNode(head, 50);
  std::cout << "50を削除後:     ";
  printList(head); // 30 -> NULL

  // リストの最後に残ったノードを解放
  deleteNode(head, 30);
}

// int main() {
//     testLinkedList();
//     return 0;
// }
```

貪欲法はソート、連結リストはポインタ操作の正確性を測る良いテストとなります。これらを踏まえ、対策を進めてください。

7. 最終チェック：ビット演算と最適化

ただし、完璧を期すために、最後にひとつだけ、**「低レベルの最適化」と「状態のコンパクトな管理」**を試すためによく出題される分野として、**ビット演算（Bitwise Operations）**をチェックリストに追加することをお勧めします。

ゲーム開発では、複数のブール値の状態（フラグ）をひとつの整数で管理したり、高速な乗除算を行ったりする際に、ビット演算が必須となります。

予想される問題

- フラグ管理 **(Bit Flags):**
  - 複数の特性（例：`CanMove`, `IsAttacking`, `IsImmune`）をひとつの `int` や `enum` で管理し、特定のフラグの **ON/OFF** や同時チェックを行うロジック。
- 高速な乗算・除算**:**
  - 2 のべき乗（例：4、8、16 など）での乗算や除算を、ビットシフト（<<, >>）で高速に実装できるか。

対策の焦点

- `&` (AND), `|` (OR), ^ (XOR), ~ (NOT), << (左シフト), >> (右シフト) の各演算子の意味と使い方を、特にフラグ管理の文脈で正確に理解すること。

C++ 実装例（ビットフラグによる状態管理）

C++

```
#include <iostream>

// フラグの定義（2 のべき乗で一意のビットを割り当てる）
enum AbilityFlags : int {
  CAN_WALK    = 1 << 0, // 0b0001 = 1
  CAN_JUMP    = 1 << 1, // 0b0010 = 2
  IS_STUNNED  = 1 << 2, // 0b0100 = 4
  IS_IMMUNE   = 1 << 3  // 0b1000 = 8
};

class CharacterAbilities {
private:
  int flags = 0; // すべての能力フラグを格納する単一の変数

public:
  // フラグをセット（ON にする）：OR 演算子 | を使う
  void setFlag(AbilityFlags flag) {
    flags |= flag;
  }

  // フラグをクリア（OFF にする）：AND 演算子 & と NOT 演算子 ~ を使う
  void clearFlag(AbilityFlags flag) {
    flags &= ~flag;
  }
```

```
  // フラグが立っているかチェック:AND 演算子 & を使う
  bool hasFlag(AbilityFlags flag) const {
    // (flags & flag) の結果が 0 でなければ、フラグは立っている
    return (flags & flag) != 0;
  }

  void test() {
    std::cout << "--- ビットフラグ管理 ---" << std::endl;

    setFlag(CAN_WALK);
    setFlag(CAN_JUMP);
    std::cout << "歩行可能? " << (hasFlag(CAN_WALK) ? "YES" : "NO") << std::endl;
// YES
    std::cout << "スタン状態? " << (hasFlag(IS_STUNNED) ? "YES" : "NO") <<
std::endl; // NO

    // スタン状態を ON にする
    setFlag(IS_STUNNED);
    // スタン中は歩行不可能にする
    clearFlag(CAN_WALK);

    std::cout << "スタン後、歩行可能? " << (hasFlag(CAN_WALK) ? "YES" : "NO") <<
std::endl; // NO

    // 複数のフラグを同時にチェック (例: スタン状態でジャンプ可能か?)
    if (hasFlag(IS_STUNNED) && hasFlag(CAN_JUMP)) {
      std::cout << ">> スタン中だがジャンプは可能" << std::endl;
    }
  }
};

// int main() {
//     CharacterAbilities c;
//     c.test();
//     return 0;
// }
```

この「ビット演算」までカバーできれば、技術的な深さ、汎用的なアルゴリズム、C++の基礎力の全てにおいて、十分すぎる対策が完了したと言えるでしょう。自信を持って試験に臨んでください。